

Aula 20: VAEs, Inferência Variacional e Reparametrização Geral

Disciplina de Aprendizado Profundo

Objetivos da Aula

- Entender modelos generativos baseados em variáveis latentes.
- Derivar a ELBO e compreender seu significado.
- Estudar reparametrização e inferência amortizada.
- Explorar variantes: β -VAE, VampPrior, IAF.
- Relacionar VAEs com métodos modernos: BBVI, ADVI.
- Construir ponte matemática para Diffusion Models.

Panorama de Modelos Generativos

- GANs: amostragem excelente, sem densidade explícita.
- Normalizing Flows: densidade exata, restrições estruturais.
- VAEs: modelo probabilístico completo, inferência aproximada.

Comparação Geral

Modelo	Densidade	Amostragem	Inferência
GAN	Não	Excelente	N/A
Flow	Sim	Boa	Trivial
VAE	Aproximada	Boa	Variacional

Por que usar VAEs?

- Modelo gerativo com estrutura explícita.
- Inferência amortizada eficiente.
- Treinamento estável e convergente.
- Ponte natural para Diffusion Models.

Modelo Latente Generativo

$$z \sim p(z) = \mathcal{N}(0, I)$$

$$x \sim p_{\theta}(x \mid z)$$

- Estrutura hierárquica: primeiro geramos z , depois x .
- A complexidade é delegada ao decoder $p_{\theta}(x \mid z)$.

O Problema da Posterior

$$p_{\theta}(z | x) = \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(x)}$$

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z)dz$$

- A integral no denominador é intratável.
- Solução: introduzir um aproximador.

Aproximação Variacional

$$q_{\phi}(z \mid x) \approx p_{\theta}(z \mid x)$$

- Parametrizamos a posterior com uma rede neural.
- Treinamos ϕ para aproximar a posterior real.

ELBO: Passo 1

$$\log p_{\theta}(x) = \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x)]$$

Adicionar e subtrair $\log q_{\phi}(z | x)$:

$$\log p_{\theta}(x) = \mathbb{E}_{q_{\phi}} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z | x)} \right] + KL(q_{\phi} \| p_{\theta})$$

ELBO: Passo 2

Separando termos:

$$\text{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - KL(q_{\phi}(z | x) \| p(z)).$$

$$\log p_{\theta}(x) = \text{ELBO}(x) + KL(q_{\phi} \| p_{\theta})$$

Como $KL \geq 0$, ELBO é bound inferior.

Interpretação da ELBO

- Termo de reconstrução:

$$\mathbb{E}_{q_{\phi}}[\log p_{\theta}(x | z)]$$

- Termo de regularização:

$$KL(q_{\phi}(z | x) \| p(z))$$

Equilíbrio entre fidelidade e simplicidade do latente.

Arquitetura Geral do VAE

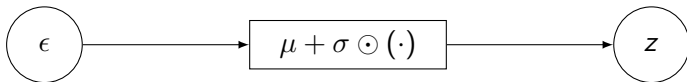


Truque da Reparametrização

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x)\epsilon, \quad \epsilon \sim N(0, I)$$

- Permite derivar gradientes de maneira estável.
- Separa incerteza (ruído) de parâmetros aprendidos.

Intuição Gráfica



KL Explícito (Gaussiano)

$$KL(q\|p) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

- Expressão fechada simples e diferenciável.

Loss do VAE

$$\mathcal{L} = -\mathbb{E}_{q_\phi}[\log p_\theta(x | z)] + KL(q_\phi(z | x) \| p(z))$$

- Reconstrução: erro entre x e $\hat{x}$.
- Regularização: aproxima posterior do prior.

Por que MNIST?

- Latente 2D fácil de visualizar.
- Dataset balanceado e simples.
- Permite analisar reconstrução, clusters e interpolação.

Encoder MLP

$$x \in \mathbb{R}^{784} \rightarrow 400 \rightarrow (\mu, \log \sigma^2)$$

- Rede simples e eficiente.
- Projection to $\mu(x)$ e $\sigma(x)$.

Decoder MLP

$$z \in \mathbb{R}^2 \rightarrow 400 \rightarrow \hat{x}$$

- Usualmente saída é Bernoulli ou logits.
- Capta estrutura dos dígitos no espaço latente.

Resumo das Seções 1–2

- VAE = modelo generativo com variáveis latentes.
- Posterior é intratável $\rightarrow$ aproximamos via q_ϕ .
- ELBO fornece objetivo de treino.
- Reparametrização dá gradientes estáveis.
- MNIST + MLP = laboratório ideal para VAEs.

Visualização do Espaço Latente 2D

- Após o treino, cada dígito ocupa regiões distintas no plano (z_1, z_2) .
- Estrutura contínua: interpolação suave entre classes.
- Permite observar agrupamentos naturais do dataset.

Interpolação no Latente

Interpolação linear entre dois códigos:

$$z(\alpha) = (1 - \alpha)z_a + \alpha z_b$$

- Produz transições suaves entre dígitos.
- Demonstra continuidade aprendida.

Reconstruções vs. Amostras

- Reconstruções mostram quão bem o modelo captura detalhes.
- Amostras do prior avaliam a capacidade generativa.

Por que β -VAE?

- Incentiva que representações latentes sejam mais independentes.
- Impõe fatoração e estrutura mais interpretável.
- Controla fluxo de informação $z \leftarrow x$.

Objetivo do β -VAE

$$\mathcal{L}_\beta = -\mathbb{E}_{q_\phi}[\log p_\theta(x|z)] + \beta KL(q_\phi(z|x) \| p(z))$$

- $\beta > 1$ força latente mais simples.
- Relação direta com Information Bottleneck.

Impacto de β no Latente

- β alto $\rightarrow$ clusters mais separados, menos variação intra-classe.
- β baixo $\rightarrow$ reconstrução mais fiel, latente menos estruturado.
- Trade-off controlado pelo usuário.

KL Annealing

Técnica para evitar posterior collapse:

$$\beta_t = \min\left(1, \frac{t}{T}\right)$$

- Começa com KL pequeno.
- Aumenta gradualmente até 1.
- Permite que rede aprenda reconstrução antes de regularizar.

Formas de Scheduler

- Linear:

$$\beta_t = \frac{t}{T}$$

- Cosseno:

$$\beta_t = \frac{1}{2}(1 - \cos(\pi t / T))$$

- Cíclico: períodos alternando aquecimento e resfriamento.

Efeito do KL Annealing

- Evita que rede ignore o latente (colapso).
- Facilita treino do decoder nas primeiras épocas.
- Conduz a representações mais limpas.

Resumo da Seção β -VAE

- β controla balanço entre reconstrução e compressão.
- Annealing melhora estabilidade do treinamento.
- Essencial para disentanglement.

Limitações do Prior Gaussiano

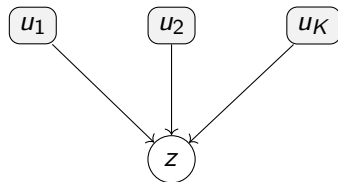
- Não se ajusta à forma real do posterior agregado.
- Pode causar mismatch entre prior e $q(z|x)$.
- Contribui para posterior collapse.

VampPrior: Definição

$$p(z) = \frac{1}{K} \sum_{k=1}^K q_{\phi}(z \mid u_k)$$

- u_k : pseudo-inputs aprendidos.
- Prior torna-se combinação da própria posterior.

Intuição do VampPrior



- Cada pseudo-input gera um modo no prior.
- Aproxima melhor distribuição real do latente.

Vantagens do VampPrior

- Evita posterior collapse.
- Prior flexível guiado pelos próprios dados.
- Aproxima “prior ótimo”: posterior agregado.

Exercício no Notebook

- Implementar prior:

$$p(z) = \frac{1}{K} \sum_k q_\phi(z | u_k)$$

- Treinar VAE com e sem VampPrior.
- Visualizar diferenças no espaço latente.

Por que usar Flows na Posterior?

- $q_{\phi}(z|x)$ gaussiano é simples demais.
- Formas complexas da posterior não são capturadas.
- Solução: aplicar normalizing flows para enriquecer q .

IAF: Equação

$$z_t = \mu_t(z_{<t}) + \sigma_t(z_{<t}) \cdot z_t$$

- Permite amostragem rápida no decoder.
- Treinável por backprop usando log-det Jacobiano.

VAE + IAF

$$q_K(z_K \mid x) = q_0(z_0 \mid x) - \sum_{k=1}^K \log |\det J_k|$$

- Encoder aprende distribuição mais expressiva.
- Decoder permanece simples.

VAE Hierárquico

$$z_2 \sim p(z_2), \quad z_1 \sim p(z_1 \mid z_2), \quad x \sim p(x \mid z_1)$$

- Múltiplos níveis capturam dependências profundas.

NVAE e Modelagem Profunda

- Redes profundas com variáveis latentes hierárquicas.
- Base para modelos de alta qualidade visual.
- Conecta-se conceitualmente a Diffusion Models.

Reparametrização Universal

Queremos expressar:

$$z \sim q_{\phi}(z) \quad \Rightarrow \quad z = g(\phi, \epsilon), \quad \epsilon \sim p(\epsilon)$$

- Generaliza reparametrização Gaussiana.
- Possibilita gradientes de baixa variância.
- Permite treinar modelos complexos de maneira estável.

Exemplos de Reparametrização

- Lognormal:

$$z = \exp(\mu + \sigma\epsilon)$$

- Gumbel:

$$g = -\log(-\log u), \quad u \sim U(0, 1)$$

- Reparametrização via CDF inversa:

$$z = F^{-1}(u), \quad u \sim U(0, 1)$$

Gumbel-Softmax para Variáveis Discretas

Reparametriza escolha discreta via relaxação contínua:

$$y = \text{softmax} \left(\frac{\log \alpha + g}{\tau} \right)$$

- g : ruído Gumbel.
- τ : temperatura.
- Permite backprop em variáveis categóricas.

Flows como Reparametrização

$$z_K = f_K \circ \cdots \circ f_1(z_0), \quad z_0 \sim N(0, I)$$

- Permite aproximar qualquer distribuição suave.
- Geral para VAEs: aplica-se ao encoder.

Black-Box Variational Inference (BBVI)

Usa identidade:

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}}[f(z)] = \mathbb{E}[f(z) \nabla_{\phi} \log q_{\phi}(z)]$$

- Não requer reparametrização explícita.
- Aplica-se a distribuições arbitrárias.
- Problema: alta variância.

Redução de Variância no BBVI

- Baselines:

$$f(z) - b$$

- Rao-Blackwellization.
- Control variates aprendidos.

ADVI: Automatic Differentiation VI

- Transforma todas variáveis para espaço real:

$$z = T^{-1}(\xi)$$

- Assume posterior variacional Gaussiana diagonal sobre ξ .
- ELBO diferenciável calculada por autodiff.

$$\xi = \mu + \sigma \epsilon$$

ADVI vs. VAE

- ADVI: inferência variacional não amortizada .
- VAE: inferência variacional amortizada via encoder.
- ADVI funciona para modelos arbitrários; VAE para dados massivos.

Comparação: BBVI, ADVI, VAE, IAF

Método	Reparam	Amortizado	Flexibilidade
BBVI	Opcional	Não	Alta
ADVI	Sim	Não	Média
VAE	Sim	Sim	Média
VAE+IAF	Sim	Sim	Alta

Diffusion Models: Ideia Fundamental

- Processo de *noising*: adiciona ruído progressivo.
- Processo de *denoising*: aprende a invertê-lo.
- Cada passo é semelhante a uma camada de VAE.

Forward Process: Encoder Fixo

$$q(x_t \mid x_0) = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$$

- Adiciona ruído controlado.
- Determina distribuição de x_t .

Reverse Process: Decoder Aprendido

$$p_{\theta}(x_{t-1} \mid x_t)$$

- Função que tenta remover ruído progressivamente.
- Parâmetros treinados minimizando divergências KL.

ELBO da Diffusion

$$\text{ELBO} = \sum_{t=1}^T \mathbb{E}_{q(x_0, x_t)} \left[KL(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)) \right]$$

- Cada termo é um mini-VAE.
- Processo inteiro = VAE com infinitas camadas.

Reparametrização na Diffusion

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, \quad \epsilon \sim N(0, I)$$

- Essencial para aprendizado via ELBO.
- Conecta-se com truque do VAE.

Score Matching em Diffusion

$$\nabla_{x_t} \log p(x_t) \approx -\epsilon_{\theta}(x_t, t)$$

- Modelo aprende gradiente da densidade.
- Torna-se possível reconstruir distribuição completa.

Comparação Estrutural: VAE vs Diffusion



- VAE: 1 camada de inferência + 1 camada de geração.
- Diffusion: dezenas ou centenas de passos variacionais.

Diffusion = VAE Profundo Contínuo

- Forward = encoder não aprendido.
- Reverse = decoder treinado.
- ELBO = soma de KLs (um por passo).

Interpretação de Alto Nível

- VAEs fornecem a teoria base (ELBO + KL + reparam).
- Diffusion expande isso para trajetória contínua.
- Score matching recupera gradiente da densidade.

Arquiteturas para Encoder e Decoder em VAEs

Nesta seção discutimos:

- MLPs, CNNs, ResNets, U-Nets;
- Autoregressive decoders e flows;
- Transformers como encoder/decoder;
- Arquiteturas hierárquicas modernas (NVAE, VDVAE);
- Relação entre arquitetura e posterior collapse.

MLPs como Encoder e Decoder

- Indicados para MNIST, dados tabulares e toy problems.
- Estrutura típica:

$$h = \sigma(W_2\sigma(W_1x))$$

- Limitados para imagens grandes.
- Bom para visualização de espaços latentes 2D.

CNN Encoders

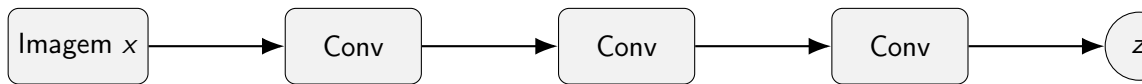
Arquitetura típica:

- Conv $\rightarrow$ Conv $\rightarrow$ Conv $\rightarrow$ Flatten $\rightarrow$ Linear $\rightarrow (\mu, \log \sigma^2)$.
- Capturam estrutura espacial local.
- Robustas e rápidas.

CNN Decoders

- Linear $\rightarrow$ reshape $\rightarrow$ ConvTranspose $\rightarrow$ ConvTranspose.
- Possibilidade de artefatos (checkerboard).
- Normalizações melhoram estabilidade.

Arquitetura CNN (Diagrama)



ResNets para VAEs

- Blocos residuais evitam vanishing gradients.
- Permitem encoders profundos, essenciais em NVAE.
- Melhora expressividade do modelo variacional.

$$h_{l+1} = h_l + F(h_l)$$

Decoders do tipo U-Net

- Skip connections mantêm informações espaciais.
- Ideais para reconstruções de alta fidelidade.
- Base de VQ-VAE-2 e Latent Diffusion.

Decoders Autoregressivos (PixelVAE)

$$p_{\theta}(x \mid z) = \prod_i p_{\theta}(x_i \mid x_{<i}, z)$$

- Capturam dependências locais fortes.
- Aumentam nitidez das imagens.
- Tradeoff: amostragem lenta.

Flow-based Decoders

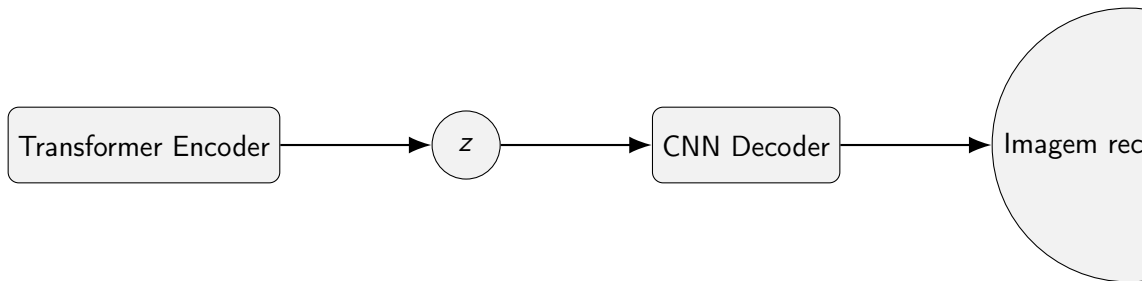
$$x = f_{\theta}(z, \epsilon), \quad \epsilon \sim p(\epsilon)$$

- Densidade explícita e exata.
- Alta expressividade.
- Custo computacional maior.

Transformers como Encoder para VAEs

- Ideais para texto e sequências.
- Usados em VAEs de linguagem e multimodais.
- Podem alimentar decoders CNN/U-Net para imagem.

Arquitetura Híbrida (TikZ)



VQ-VAE (Vector Quantized VAE)

- Latentes discretos: dicionário de embeddings.
- Usado em Stable Diffusion, DALL-E, Imagen.
- Reduz dimensionalidade antes de diffusion.

VAEs Hierárquicos (NVAE, VDVAE)

- Priors em múltiplos níveis.
- Upsampling e downsampling progressivo.
- Melhor manejo de detalhes e estruturas globais.

Resumo Prático: Qual Arquitetura Usar?

- Encoder deve ser mais forte que decoder → evita collapse.
- Skip connections ajudam em reconstrução.
- Autoregressive decoders melhoram qualidade, mas são lentos.
- Transformers úteis para texto e multimodal.

VAEs e SAEs para Circuit Discovery em LLMs

Objetivos:

- entender superposição e features latentes;
- extrair representações interpretáveis;
- usar SAEs e SVAEs para revelar circuitos internos;
- aplicações práticas em alignment, steering e edição.

Representações Internas em LLMs

- Vetores de alta dimensão, densos e polisemânticos.
- Mecanismo de superposição: várias features por neurônio.
- Necessidade de decomposição não-linear e esparsa.

O Problema da Superposição

Features excedem capacidade de neurônios:

$$x = \sum_k \alpha_k v_k$$

Implicações:

- neurônios representam múltiplas features;
- torna interpretabilidade difícil;
- SAEs foram criados para “desembaralhar” features.

Sparse Autoencoders (SAEs)

$$z = \text{ReLU}(Wx + b)$$

$$\hat{x} = W'z$$

Loss:

$$L = \|x - \hat{x}\|^2 + \lambda \|z\|_1$$

- Produzem features esparsas e interpretáveis.
- Base das interpretações modernas de LLMs.

Sparse Variational Autoencoders (SVAE)

$$q_{\phi}(z | x) = \mathcal{N}(\mu(x), \sigma(x))$$

Sparsidade via prior:

$$p(z) = \text{Laplace, Spike-and-slab, Logit-normal}$$

Vantagens:

- interpretabilidade + incerteza;
- separação mais limpa de circuitos;
- melhor robustness.

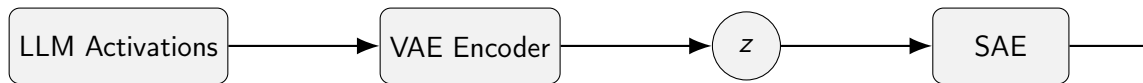
Comparação: SAE, VAE, SVAE

Modelo	Probabilístico	Esparso	Interpretável
VAE	sim	não	médio
SAE	não	sim	alto
SVAE	sim	sim	máximo

Pipeline para Descoberta de Circuitos

- 1 Extrair ativações de um LLM.
- 2 Treinar VAE para compressão global.
- 3 Treinar SAE sobre o espaço latente ou diretamente nas ativações.
- 4 Obter features esparsas interpretáveis.
- 5 Analisar circuitos, steering vectors e intervenções.

Diagrama: VAE + SAE



Features interpretáveis em SAEs

SAEs revelam:

- sintaxe e estrutura frasal;
- tracking de entidades;
- mecanismos de indução;
- bias e comportamentos emergentes;
- "neurônios de moralidade" e segurança.

De Polisemântico a Monosemântico

SAEs transformam:

um neurônio polisemântico $\rightarrow$ várias features monosemânticas

Resultado:

- maior interpretabilidade;
- vetores de steering claros;
- decomposição causal mais limpa.

SVAE: Papel do Prior Esparso

- controla a dimensão efetiva do latente;
- evita reconstruções difusas;
- fornece incerteza explícita das features;
- mais robusto a ruído e drift.

Aplicações: Steering e Edição Comportamental

Manipular o latente do SAE ou SVAE permite:

- controlar formalidade, toxicidade, profundidade lógica;
- remover ou ativar bias;
- reforçar consistência factual;
- edição de estilo e persona.

Aplicações: Circuit Patching e Causal Tracing

- Intervir em componentes específicos de z ;
- avaliar impacto causal no output do LLM;
- identificar caminhos de raciocínio internos.

Exemplo Real: Feature de Moralidade

SAEs treinados sobre Llama-3 revelam features como:

- julgamento moral implícito;
- preferências civilizacionais;
- mecanismos de recusa.

Manipular a feature altera o comportamento.

VAE + SAE = Framework Completo

- VAE: estrutura contínua, amostragem, compressão.
- SAE: interpretabilidade extrema.
- SVAE: integração probabilística, prior esperso.

Exercício 1: Treinar VAE CNN no MNIST

- Implementar encoder conv + decoder conv.
- Usar latente de 2 dimensões.
- Fazer scatter-plot do espaço latente.
- Mostrar reconstruções.

Exercício 2: Adicionar β -VAE

- substituir KL por βKL ;
- variar $\beta \in \{0.1, 1, 5, 10\}$;
- medir separabilidade das classes no latente.

Exercício 3: VampPrior

- implementar prior:

$$p(z) = \frac{1}{K} \sum_k q_\phi(z | u_k)$$

- testar com $K = 10$;
- observar impacto na KL.

Exercício 4: SAE em embeddings de LLM

- coletar embeddings de uma LLM pequena;
- treinar SAE com esparsidade moderada;
- identificar features interpretáveis;
- manipular features e medir efeito no output.

Resumo da Seção

- VAE modela estrutura global.
- SAE descobre circuitos interpretáveis.
- SVAE une interpretabilidade + incerteza.
- Integração com LLMs é estado da arte em interpretabilidade.

Próxima Aula: Diffusion Models em Profundidade

- DDPM, score matching.
- U-Nets condicionados em tempo.
- Sampling, treinamento e ELBO da diffusion.
- Conexões com flows, VAEs e energia.

Resumo Final da Aula

- VAEs: modelo generativo latente com ELBO.
- Extensões: β -VAE, VampPrior, IAF, hierárquicos.
- Reparametrização universal e métodos variacionais.
- Diffusion Models reinterpretados como VAEs infinitos.

Perguntas?